

Техническое задание: Приложение для управления заказами

Описание проекта

Разработать REST API приложение на Spring Boot для управления заказами и клиентами. Приложение должно демонстрировать навыки работы с **Spring REST API**, **ORM (Hibernate)**, а также учитывать потенциальные проблемы

Бизнес-требования

1. Обязательно - управление клиентами:

- Создание, просмотр, редактирование клиентов
 - Обязательные поля: имя, email (уникальное)
 - Поиск клиентов по ключевому слову (поиск по имени, email, адресу и т.д - частичное совпадение от 3х символов, регистронезависимость)
- Не удалять клиента, только отметка как активный/неактивный (время когда стал неактивен)

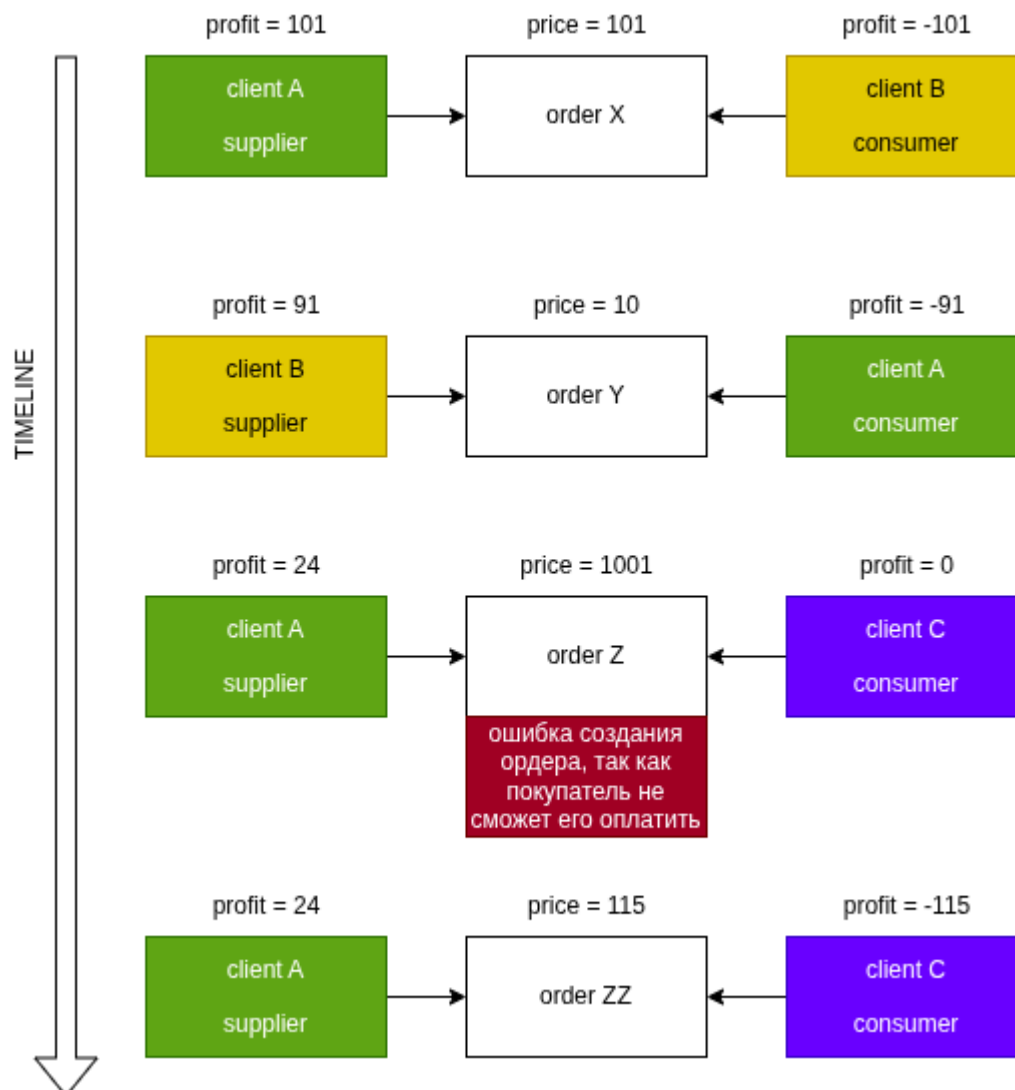
2. Обязательно - управление заказами:

- Создание, просмотр заказов
 - Обязательные поля: название, поставщик, потребитель, цена, время старта обработки, время окончания обработки, время сохранения в бд
 - Бизнес-ключ: название + поставщик + потребитель
 - При инициации заказа эмулировать задержку обработки (случайная пауза от 1 до 10 секунд) перед записью в БД
- запрещено создавать заказ с 0 и отрицательно ценой
- запрещено инициировать заказ, если суммарная выгода клиента станет меньше -1000
- запрещено создавать заказ для неактивного клиента

3. Обязательно

- Просмотр всех заказов покупки/продажи конкретного клиента (поставщик или потребитель)
- Вывод суммарной выгоды клиента по всем его заказам
 - **выгода клиента** - сумма цены сделок, для продавца положительная, для покупателя отрицательная
- Поиск клиентов в диапазоне выгоды по заказам

Пример бизнес флоу



Возможные сценарии

1. пользователь по ошибке отправляет на создание $N + 1$ одинаковых заказов с ценой 1. Должен инициироваться/создаться только один валидный заказ - остальные возвращают ошибку
2. пользователь по ошибке отправляет на создание 10 одинаковых заказов с уменьшающейся ценой от 100 до 10 с шагом 10. у покупателя выгода на момент отправки -970, должен инициироваться/создаться только один валидный заказ
3. пользователь отправляет на создание $N + 1$ разных заказов, в этот же момент клиента делают неактивным (тоже через api). должны создаваться только заказы которые были обработаны до момента когда клиент стал неактивен

Технические требования

1. **Технологии:**
 - Использовать **Spring Boot** для создания REST API
 - Использовать **Hibernate (Spring Data JPA)** для работы с сущностями
 - Использовать докер и докер компоуз
 2. **База данных:**
 - Использовать реляционную БД (**PostgreSQL/H2**).
 - Реализовать связи Client (как поставщик/потребитель) и Order
 3. **API endpoints:**
 - реализовать рест эндпоинты покрывающие **бизнес требования**
 4. **Ожидания от приложения:**
 - Реализован обязательный функционал бизнес-флоу на стороне бэкенда
 - Все сценарии должны быть выполнимы в готовом приложении
 - Реализована автоматизация двух любых сценариев из раздела "Возможные сценарии" (отдельные приложения/методы работающее с апи основного приложения)
 5. **Требования к коду:**
 - Обработка ошибок (например, клиент не найден, конфликт бизнес-ключа и т.д).
 - Логирование ключевых операций и ошибок
 - Минимальный UI (например, Swagger) для тестирования API
 - Инструкция настройки и запуска приложения (README или INSTALL)
-

Опционально

1. реализовать полную отдельную имитацию всех сценариев - отдельное приложение работающее с апи основного приложения
2. реализовать метод обнуляющий профит всех клиентов через имеющийся апи (в любой момент профит каждого клиента при вызове метода становится 0) - отдельное приложение работающее с апи основного приложения
3. реализовать квери обнуляющий профит всех клиентов без удаления/редактирования имеющихся ордеров (в любой момент профит каждого клиента при вызове метода становится 0) - sql квери на INSERT